

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
ERROR HANDLING**



Disusun oleh :

Naufal Fakhrianto Nugroho 105224026

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026**

BAB I

PENDAHULUAN

Praktikum Pemrograman Berorientasi Objek pada Modul 11 ini membahas konsep Error Handling sebagai mekanisme penanganan kesalahan yang robust dalam aplikasi Java. Sebuah program yang baik tidak hanya berjalan benar pada kondisi normal, tetapi juga mampu bertahan dan memberikan respons yang informatif ketika menghadapi kondisi tak terduga seperti input tidak valid, data tidak ditemukan, atau keterbatasan sumber daya.

Studi kasus yang diimplementasikan adalah Sistem Reservasi Tiket Kereta Api berbasis (Command Line Interface) bernama JAVA EXPRESS. Program ini mengintegrasikan tiga mekanisme error handling utama Java: blok try-catch-finally untuk menangkap exception, keyword throw untuk melempar exception secara eksplisit, keyword throws untuk mendeklarasikan potensi exception pada signature method, serta tiga buah Custom Exception yang dirancang sesuai konteks domain bisnis sistem reservasi.

BAB II

VARIABLE

No	Nama Variable	Tipe Data	Fungsi
1	kodeKereta	String	Penyimpan kode unik untuk identitas kereta
2	namaKereta	String	Penyimpan nama kereta
3	rute	String	Penyimpan rute perjalanan kereta
4	sisakursi	int	Penyimpan jumlah kursi yang masih ada
5	daftarKereta	Map < String, Kereta >	Basis data berbasis HashMap key value
6	nikInput	String	Penyimpan Nik asli menyebabkan Exception
7	kodeInput	String	Penyimpan kode

			kereta yang tidak ada
8	namaKereta*	String	Atribut dari TiketHabisException : nama keretanya
9	sisaKursi*	Int	Atribut dari TiketHabisException : sisa kursi yang tersedia
10	berjalan	boolean	Flag kendali loop utama menu CLI
11	pilihan	int	Buat nampung pilihan dari pengguna
12	kode	String	Inputan kode kereta dari pengguna saat pemesanan
13	nik	String	Input NIK penumpang dari pengguna
14	nama	String	Input nama penumpang dari pengguna
15	jumlah	int	Input jumlah tiket yang dipesan pengguna

BAB III

CONSTRUCTOR dan METHOD

No	Nama Method	Jenis Method	Fungsi
1	Kereta()	Constructor	Menginisialisasi objek kereta dengan kode, nama, rute, dan kursi
2	kurangiKursi(int jumlah)	Functional	Mengurangi jumlah

			sisa kursi setelah pemesanan berhasil
3	DataPenumpangTidakValidException()	Constructor	Inisialisasi Unchecked Exception dengan pesan dan NIK input
4	getNikInput()	Functional	Getter untuk mengambil NIK yang menyebabkan exception
5	RuteTidakDitemukanException()	Constructor	Inisialisasi Checked Exception dengan pesan dan kode input
6	getKodeInput()	Functional	Getter untuk mengambil kode kereta yang tidak ditemukan
7	TiketHabisException()	Constructor	Inisialisasi Checked Exception dengan pesan, nama kereta, sisa kursi
8	getNamaKereta()	Functional	Getter nama kereta dari TiketHabisException
9	getSisaKursi()	Functional	Getter sisa kursi dari TiketHabisException
10	SistemReservasi()	Constructor	Menginisialisasi HashMap dan mendaftarkan 2 kereta awal
11	tambahKereta()	Procedural	Mendaftarkan objek kereta ke dalam HashMap
12	pesanTiket()	Procedural	Memproses reservasi dengan 3 lapis validasi dan throws
13	main()	Procedural	Entry point: menu

			CLI dengan loop dan try-catch-finally
14	prosesPemesanan()	Procedural	Menangani input form dan menangkap semua exception pemesanan

BAB IV DOKUMENTASI dan PEMBAHASAN

A. Class DataPenumpangTidakValidException (Unchecked Exception)

```

1  public class DataPenumpangTidakValidException extends RuntimeException {
2
3      private String nikInput;
4
5      public DataPenumpangTidakValidException(String pesan, String nikInput) {
6          super(pesan);
7          this.nikInput = nikInput;
8      }
9
10     public String getNikInput() {
11         return nikInput;
12     }
13 }

```

- Line 1 'public class DataPenumpangTidakValidException extends RuntimeException': Kata kunci extends RuntimeException menjadikan kelas ini sebagai Unchecked Exception. Berbeda dengan Checked Exception yang mengharuskan pemanggil mendeklarasikan atau menangani exception, Unchecked Exception tidak wajib ditulis di signature method manapun. Ini dipilih karena validasi NIK yang gagal adalah akibat input pengguna yang tidak terduga dan tidak perlu dipaksakan untuk selalu ditangani.
- Line 3 'private String nikInput': Deklarasi atribut privat bertipe String untuk menyimpan nilai NIK asli yang dikirimkan pengguna saat exception dilempar. Atribut ini menjadi pembeda utama dari exception biasa dengan menyimpan nilai NIK yang bermasalah, blok catch dapat mencetak informasi detail ('NIK yang Anda masukkan: ...') kepada pengguna tanpa perlu menyusun ulang pesan dari nol.
- Line 5 Konstruktor menerima dua parameter: 'pesan' bertipe String berisi keterangan kesalahan, dan 'nikInput' bertipe String berisi NIK yang menyebabkan kegagalan. Dua parameter ini memisahkan tanggung jawab:

pesan untuk ditampilkan via getMessage(), dan nikInput untuk ditampilkan via getter khusus.

- Line 6 'super(pesan)': Memanggil konstruktor kelas induk RuntimeException dengan meneruskan parameter pesan. Pemanggilan super() ini wajib dilakukan agar mekanisme getMessage() bawaan Java dapat bekerja dengan benar — tanpa ini, e.getMessage() akan mengembalikan null saat exception ditangkap di blok catch.
- Line 7 'this.nikInput = nikInput': Menyimpan nilai NIK ke atribut kelas menggunakan kata kunci this untuk membedakan antara atribut kelas (this.nikInput) dengan parameter konstruktor lokal (nikInput) yang memiliki nama identik.
- Line 10 'public String getNikInput()': Getter yang memungkinkan blok catch mengakses nilai NIK yang menyebabkan exception setelah objek exception ditangkap. Tanpa getter ini, informasi NIK akan tersembunyi di dalam objek exception dan tidak bisa diambil.

B. Class RuteTidakDitemukanException (Checked Exception)

```
PraktikumWeek11 > THT > src > RuteTidakDitemukanException.java > RuteTidakDitemukanException
1  public class RuteTidakDitemukanException extends Exception {
2
3      private String kodeInput;
4
5      public RuteTidakDitemukanException(String pesan, String kodeInput) {
6          super(pesan);
7          this.kodeInput = kodeInput;
8      }
9
10     public String getKodeInput() {
11         return kodeInput;
12     }
13 }
```

- Line 1 'extends Exception': Menjadikan kelas ini sebagai Checked Exception. Pilihan ini disengaja karena kondisi 'kode kereta tidak ditemukan' adalah skenario bisnis yang dapat diprediksi dan wajib ditangani secara eksplisit. Akibat extends Exception ini, kompiler Java akan memaksa setiap method yang melempar exception ini untuk mendeklarasikan 'throws RuteTidakDitemukanException' di signature-nya, dan setiap pemanggil method tersebut wajib memiliki blok catch yang sesuai.
- Line 3 'private String kodeInput': Menyimpan string kode kereta yang dimasukkan pengguna namun tidak ditemukan di HashMap.
- Line 6 'super(pesan)': Sama seperti exception sebelumnya, memanggil konstruktor Exception agar getMessage() dapat bekerja.

- Line 10 'public String getKodeInput()': Getter untuk mengekspos nilai kodeInput ke luar objek exception. Di blok catch kelas Main, getter ini dipanggil untuk mencetak kode yang salah kepada pengguna.

C. Class TiketHabisException

```
PraktikumWeek11 > THT > src > TiketHabisException.java > TiketHabisException
1 public class TiketHabisException extends Exception {
2
3     private String namaKereta;
4     private int sisaKursi;
5
6     public TiketHabisException(String pesan, String namaKereta, int sisaKursi) {
7         super(pesan);
8         this.namaKereta = namaKereta;
9         this.sisaKursi = sisaKursi;
10    }
11
12    public String getNamaKereta() {
13        return namaKereta;
14    }
15
16    public int getSisaKursi() {
17        return sisaKursi;
18    }
19 }
```

- Line 1 extends Exception: Checked Exception, sama seperti RuteTidakDitemukanException. Kondisi kursi habis adalah skenario bisnis yang dapat diprediksi sehingga wajib ditangani secara eksplisit oleh pemanggil.
- Line 3-4 : Dua atribut privat: namaKereta (String) dan sisaKursi (int). TiketHabisException sengaja dirancang membawa dua data kontekstual tambahan di luar pesan teks biasa. Ini adalah implementasi best practice custom exception: exception bukan hanya sinyal error, melainkan juga pembawa data domain yang relevan agar blok catch dapat memberikan respons yang sangat informatif kepada pengguna.

- Line 6 : Konstruktor dengan tiga parameter: 'pesan' untuk getMessage(), namaKereta untuk menginformasikan kereta mana yang kehabisan kursi, dan sisaKursi bertipe int untuk menampilkan angka pasti kursi yang tersisa. Dengan tiga parameter ini, blok catch bisa menampilkan pesan seperti: 'Kereta Parahyangan hanya memiliki 15 kursi tersisa, Anda memesan 20 kursi'.
- Line 8-9 : this.namaKereta = namaKereta dan this.sisaKursi = sisaKursi: Kedua atribut diinisialisasi dari parameter konstruktor. Pemakaian this. wajib karena nama parameter dan nama atribut kelas identik sehingga tanpa this, Java akan mengasumsikan keduanya adalah variabel lokal yang sama.
- Line 12-16 : Dua getter: getNamaKereta() dan getSisaKursi(). Di blok catch kelas Main, keduanya dipanggil untuk mencetak detail kereta dan sisa kursi secara terpisah: .getNamaKereta() dan .getSisaKursi().

D. Class Kereta

```

PraktikumWeek11 > THT > src > Kereta.java > Kereta
1  public class Kereta {
2      private String kodeKereta;
3      private String namaKereta;
4      private String rute;
5      private int    sisaKursi;
6
7      public Kereta(String kodeKereta, String namaKereta, String rute, int sisaKursi) {
8          this.kodeKereta = kodeKereta;
9          this.namaKereta = namaKereta;
10         this.rute       = rute;
11         this.sisaKursi  = sisaKursi;
12     }
13
14     public String getKodeKereta(){
15         return kodeKereta;}
16
17     public String getNamaKereta(){
18         return namaKereta;}
19
20     public String getRute(){
21         return rute; }
22
23     public int getSisaKursi(){
24         return sisaKursi; }
25
26
27     public void kurangiKursi(int jumlah) {
28         this.sisaKursi -= jumlah;
29     }
30 }

```

- Line 2–5 Empat atribut private: kodeKereta, namaKereta, rute (semuanya String), dan sisaKursi (int). Seluruh atribut dideklarasikan private sesuai prinsip enkapsulasi OOP tidak ada kelas lain yang bisa mengubah data kereta secara langsung tanpa melalui method resmi yang disediakan.

- Line 7–12 : Konstruktor menerima empat parameter sekaligus. Semua atribut diinisialisasi dalam satu konstruktor ini menggunakan kata kunci `this` untuk membedakan parameter lokal dari atribut kelas. Ini memastikan setiap objek `Kereta` selalu tercipta dalam kondisi lengkap, tidak pernah setengah terisi.
- Line 14-23 Empat getter read-only: `getKodeKereta()`, `getNamaKereta()`, `getRute()`, dan `getSisaKursi()`. Seluruhnya hanya mengembalikan nilai tanpa mengizinkan modifikasi, sehingga data kereta aman dari perubahan tidak sah. Tidak ada setter untuk `kodeKereta`, `namaKereta`, dan `rute` karena data tersebut tidak boleh berubah setelah kereta dibuat.
- Line 27 `public void kurangiKursi(int jumlah)` : Satu-satunya method yang mengizinkan perubahan state objek `Kereta`. Method ini hanya dipanggil oleh `SistemReservasi` setelah semua validasi lolos, memastikan pengurangan kursi tidak pernah terjadi dalam kondisi yang tidak valid.

E. Class SistemReservasi

```
PraktikumWeek11 > THT > src > SistemReservasi.java > SistemReservasi > daftarKereta
1  import java.util.HashMap;
2  import java.util.Map;
3  public class SistemReservasi {
4
5      private Map<String, Kereta> daftarKereta;
6
7      public SistemReservasi() {
8          daftarKereta = new HashMap<>();
9          tambahKereta(new Kereta(kodeKereta: "K01", namaKereta: "Argo Bromo", rute: "JKT
10         tambahKereta(new Kereta(kodeKereta: "K02", namaKereta: "Parahyangan", rute: "JKT
11     }
12     private void tambahKereta(Kereta k) {
13         daftarKereta.put(k.getKodeKereta(), k);
14     }
15     public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jum
16         throws RuteTidakDitemukanException, TiketHabisException {
17
18         if (nik.length() != 16) {
19             throw new DataPenumpangTidakValidException(
20                 "NIK harus tepat 16 karakter! Input anda: " + nik.length() + " karakter
21         }
22         for (char c : nik.toCharArray()) {
23             if (!Character.isDigit(c)) {
24                 throw new DataPenumpangTidakValidException(
25                     pesan: "NIK hanya boleh berisi angka, bukan huruf atau simbol!", nik
26             }
27         }
28         Kereta kereta = daftarKereta.get(kodeKereta.toUpperCase());
29         if (kereta == null) {
30             throw new RuteTidakDitemukanException(
31                 "Kode kereta '" + kodeKereta + "' tidak ditemukan dalam sistem.", kodeK
32         }
33     }
```

```

33     if (jumlahTiket > kereta.getSisaKursi()) {
34         throw new TiketHabisException(
35             "Kursi tidak mencukupi untuk kereta " + kereta.getNamaKereta() + ".",
36             kereta.getNamaKereta(),
37             kereta.getSisaKursi());
38     }
39     kereta.kurangiKursi(jumlahTiket);
40     System.out.println(x: "TIKET BERHASIL DIPESAN!");
41     System.out.printf (format: "Penumpang   : ", namaPenumpang);
42     System.out.printf (format: "NIK       : ", nik);
43     System.out.printf (format: "Kereta    : ", kereta.getNamaKereta());
44     System.out.printf (format: "Rute      : ", kereta.getRute());
45     System.out.printf (format: "Jumlah    : ", jumlahTiket + " tiket");
46     System.out.printf (format: "Sisa Kursi : ", kereta.getSisaKursi() + " kursi");
47 }
48

```

- Line 1–2 Import java.util.HashMap dan java.util.Map: Kedua import ini diperlukan karena tipe data HashMap dan Map bukan bagian dari package java.lang yang otomatis tersedia. Map dipakai sebagai tipe deklarasi atribut (Programming to Interface), sedangkan HashMap dipakai sebagai implementasi konkret saat instansiasi.
- Line 5 private Map<String, Kereta> daftarKereta: Deklarasi HashMap dengan generics. String sebagai tipe kunci (key) berarti pencarian dilakukan berdasarkan kode kereta seperti 'K01'. Kereta sebagai tipe nilai (value) berarti setiap kunci memetakan ke objek Kereta lengkap. Akses modifier private memastikan HashMap ini hanya bisa dimanipulasi melalui method resmi kelas SistemReservasi.
- Line 8 daftarKereta = new HashMap(): Instansiasi HashMap kosong menggunakan diamond operator yang secara otomatis menyimpulkan tipe generics dari deklarasi di Line 5. HashMap dipilih karena kompleksitas pencarian O(1) yang sangat cepat untuk validasi kode kereta.
- Line 9–10 Dua pemanggilan tambahKereta(): Data awal kereta dimasukkan langsung di konstruktor sehingga sistem selalu memiliki data saat pertama dijalankan.
- Line 12-14 Method private tambahKereta(): Akses modifier private berarti method ini hanya bisa dipanggil dari dalam kelas SistemReservasi sendiri. Baris 14 memanggil daftarKereta.put() dengan kunci k.getKodeKereta() (String seperti 'K01') dan nilai k (objek Kereta). put() menyimpan pasangan kunci-nilai ke HashMap.
- Line 15 - 16 : throws RuteTidakDitemukanException, TiketHabisException Deklarasi throws di signature method adalah kewajiban untuk Checked Exception. Dua nama exception dipisahkan koma. Perhatikan DataPenumpangTidakValidException TIDAK ditulis di sini karena sifatnya Unchecked (extends RuntimeException) sehingga tidak perlu dideklarasikan, walaupun method ini juga bisa melemparnya.
- Line 18 if (nik.length() != 16) { : Method length() pada String mengembalikan jumlah karakter. Kondisi != 16 mendeteksi NIK yang terlalu pendek maupun

terlalu panjang. Validasi ini diletakkan pertama sebelum cek karakter agar tidak terjadi iterasi sia-sia pada NIK yang panjangnya sudah salah.

- Line 19 `throw new DataPenumpangTidakValidException` : Keyword `throw` secara eksplisit melempar objek exception yang baru dibuat (`new`). Objek ini langsung dikirim ke runtime Java untuk diteruskan ke atas call stack hingga ditemukan blok `catch` yang sesuai. Pesan error menyertakan `nik.length()` agar pengguna tahu berapa karakter yang mereka masukkan.
- Line 22 `for (char c : nik.toCharArray()) {` : Method `toCharArray()` mengonversi String NIK menjadi array karakter `char`. For-Each loop kemudian memeriksa setiap karakter satu per satu. Ini diperlukan karena tidak ada method bawaan Java yang langsung mengecek apakah seluruh karakter string adalah digit.
- Line 23 `if (!Character.isDigit(c)) {` : method statis dari kelas `Character` bawaan Java yang mengembalikan `true` jika karakter `c` adalah angka 0–9. Operator `!` (negasi) membalikinya: jika karakter BUKAN angka, kondisi `true` dan exception dilempar. Loop langsung berhenti setelah `throw` pertama.
- Line 28 `daftarKereta.get(kodeKereta.toUpperCase())`: Method `toUpperCase()` mengonversi input pengguna ke huruf kapital sebelum dicari di `HashMap`. Ini membuat pencarian case-insensitive pengguna bisa mengetik 'k01' atau 'K01', keduanya akan cocok dengan kunci 'K01' di `HashMap`. Hasil `get()` disimpan ke variabel referensi bertipe `Kereta`.
- Line 33 `if (jumlahTiket > kereta.getSisaKursi())`: Membandingkan jumlah tiket yang diminta dengan sisa kursi yang tersedia di objek `Kereta`. Validasi ini hanya sampai ke baris ini jika kode kereta sudah terbukti valid (baris 20 lolos), sehingga pemanggilan `kereta.getSisaKursi()` dijamin aman dari `NullPointerException`.

F. Class Main

```

1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  public class Main {
5
6      Run | Debug
7      public static void main(String[] args) {
8          Scanner scanner = new Scanner(System.in);
9          SistemReservasi reservasi = new SistemReservasi();
10
11          System.out.println(x: " SELAMAT DATANG DI JAVA EXPRESS  ");
12          System.out.println(x: " Sistem Reservasi Tiket Kereta Api ");
13
14          boolean berjalan = true;
15
16          while (berjalan) {
17              System.out.println(x: "\nMENU UTAMA");
18              System.out.println(x: " 1. Pesan Tiket");
19              System.out.println(x: " 2. Keluar");
20              System.out.print(s: "Pilih menu (1-2): ");
21
22              try {
23                  int pilihan = scanner.nextInt();
24                  scanner.nextLine();
25
26                  switch (pilihan) {
27                      case 1:
28                          prosesPemesanan(scanner, reservasi);
29                          break;
30
31                      case 2:
32                          berjalan = false;
33                          break;
34
35                      default:
36                          System.out.println(x: "Pilihan tidak valid. Masukkan angka 1, atau 2 ");
37                  }
38              } catch (InputMismatchException e) {
39                  System.out.println(x: "ERROR: Input menu harus berupa angka, bukan teks!");
40                  scanner.nextLine();
41              }
42          }
43
44          try {
45              System.out.println(x: "\n Terima kasih telah menggunakan JAVA EXPRESS!");
46              System.out.println(x: " Sampai jumpa dan selamat bepergian!");
47          } finally {
48              scanner.close();
49              System.out.println(x: "[Sistem] Scanner ditutup. Sesi berakhir.");
50          }
51
52          private static void prosesPemesanan(Scanner scanner, SistemReservasi reservasi) {
53              try {
54                  System.out.println(x: "\n FORM PEMESANAN TIKET ");
55
56                  System.out.print(s: "Kode Kereta (K01/K02): ");
57                  String kode = scanner.nextLine().trim();
58
59                  System.out.print(s: "NIK Penumpang (16 digit): ");
60                  String nik = scanner.nextLine().trim();
61
62                  System.out.print(s: "Nama Penumpang: ");
63                  String nama = scanner.nextLine().trim();
64
65                  System.out.print(s: "Jumlah Tiket: ");
66                  int jumlah = scanner.nextInt();
67                  scanner.nextLine();

```

```

67     reservasi.pesanTiket(kode, nik, nama, jumlah);
68
69     } catch (InputMismatchException e) {
70         System.out.println(x: "ERROR: Jumlah tiket harus berupa angka!");
71         scanner.nextLine();
72     } catch (DataPenumpangTidakValidException e) {
73         System.out.println("DATA TIDAK VALID: " + e.getMessage());
74         System.out.println("NIK yang Anda masukkan: " + e.getNikInput());
75     } catch (RuteTidakDitemukanException e) {
76         System.out.println("RUTE TIDAK DITEMUKAN: " + e.getMessage());
77         System.out.println("Kode yang dimasukkan: " + e.getKodeInput());
78     } catch (TiketHabisException e) {
79         System.out.println("TIKET TIDAK TERSEDIA: " + e.getMessage());
80         System.out.println("Kereta : " + e.getNamaKereta());
81         System.out.println("Sisa kursi: " + e.getSisaKursi() + " kursi");
82     }
83 }
84 }

```

- Loop utama while(berjalan) dibungkus try-catch InputMismatchException. Ini menangani input non-angka saat memilih menu (1/2/3). scanner.nextLine() setelah catch adalah KRITIS — tanpa ini, nilai invalid tetap di buffer dan program terjebak infinite loop karena scanner.nextInt() terus membaca nilai yang sama tanpa berhenti.
- Method prosesPemesanan() memiliki satu blok try dengan empat blok catch yang menangkap exception secara berurutan dari paling spesifik ke paling umum: InputMismatchException (input jumlah tiket).
- Blok try-finally di akhir main() memastikan scanner.close() selalu dieksekusi saat pengguna memilih keluar, bahkan jika terjadi exception tak terduga sebelumnya. Ini mengikuti best practice pembersihan resource di blok finally.
- Setiap catch memanfaatkan getter dari custom exception (getNikInput(), getKodeInput(), getNamaKereta(), getSisaKursi()) untuk menampilkan pesan error yang ramah dan informatif, bukan hanya pesan generik.

BAB V

KESIMPULAN

Berdasarkan implementasi Sistem Reservasi Tiket JAVA EXPRESS, dapat disimpulkan bahwa mekanisme Error Handling Java memberikan kontrol penuh atas kondisi-kondisi abnormal yang mungkin terjadi selama eksekusi program tanpa menyebabkan crash.

Penggunaan Custom Exception yang domain-spesifik (Data Penumpang Tidak Valid Exception,,Rute Tidak Ditemukan Exception,Tiket Habis Exception) terbukti menghasilkan kode yang lebih ekspresif dan pesan error yang lebih informatif dibandingkan menggunakan exception generik bawaan Java. Perbedaan antara Checked dan Unchecked Exception juga diterapkan secara tepat Checked untuk kondisi yang dapat diprediksi dan wajib ditangani (rute tidak ada, tiket habis), dan Unchecked untuk kesalahan programming/input yang tidak terduga (NIK tidak valid).

Blok try-catch berlapis di kelas Main berhasil memastikan program stabil menghadapi berbagai jenis input salah, sementara blok finally menjamin pembersihan resource Scanner selalu dilakukan. Pola scanner.nextLine() setelah catch InputMismatchException terbukti krusial untuk mencegah infinite loop akibat buffer yang tidak dibersihkan.

BAB VI

DAFTAR PUSTAKA

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemrograman Berorientasi Objek: Java Collections*. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.